❸. If you get to the end of the list without finding a free entry ❹, there are too many threads using hazard pointers, so you throw an exception.

Once the `hp_owner` instance has been created for a given thread, further accesses are much faster because the pointer is cached, so the table doesn't have to be scanned again.

When each thread exits, if an instance of `hp_owner` was created for that thread, then it's destroyed. The destructor then resets the actual pointer to `nullptr` before setting the owner ID to `std::thread::id()`, allowing another thread to reuse the entry later ❺.

With this implementation of `get_hazard_pointer_for_current_thread()`, the implementation of `outstanding_hazard_pointers_for()` is really simple: just scan through the hazard pointer table looking for entries:

```
bool outstanding_hazard_pointers_for(void* p)
{
    for(unsigned i=0;i<max_hazard_pointers;++i)
    {
        if(hazard_pointers[i].pointer.load()==p)
         {
            return true;
         }
    }
    return false;
}
```

It's not even worth checking whether each entry has an owner: unowned entries will have a null pointer, so the comparison will return `false` anyway, and it simplifies the code.

`reclaim_later()` and `delete_nodes_with_no_hazards()` can then work on a simple linked list;

`reclaim_later()` just adds nodes to the list, and `delete_nodes_with_no_hazards()` scans through the list, deleting